Tareas 5 – pruebas automatizadas, integración, regresión y funcionales

Pedro Julio Caso – 241286

Diego Andre Calderón – 241263

Javier Sebastián Alvarado Monzón – 24546

Hugo Méndez Lee – 241265

José Miguel Rosas Guerra – 241274

Ingeniería de Software II

Sección 30

Universidad del Valle de Guatemala

Fecha: 6/10/2026

Herramientas de integración y regresión

1. **Playwright para pruebas de regresión, Testcontainers, vitest, supertest para integración backend:**
   Esta combinación permite automatizar pruebas de integración y regresión en todo el sistema. Playwright verifica los flujos principales desde la interfaz web en navegadores reales, mientras que Vitest y Supertest prueban los endpoints de Express. Testcontainers complementa estas pruebas creando una instancia temporal de MariaDB, lo que permite validar consultas, transacciones y persistencia real sin afectar la base de datos de desarrollo o producción.

2. **Cypress para pruebas de regresión, Docker Compose, Supertest para pruebas de integración:**
   Cypress permite automatizar los flujos principales desde la interfaz web y detectar si un cambio afecta funcionalidades existentes. Docker Compose crea un entorno controlado con frontend, backend y MariaDB, mientras Supertest verifica la comunicación entre las rutas, middlewares y controladores de Express mediante solicitudes HTTP.

3. **Postman CLI para pruebas de regresión de la API, PactJS, Pact Broker para pruebas de integración:**
   Postman CLI ejecuta colecciones automatizadas para comprobar que los endpoints mantengan sus códigos de estado, respuestas y reglas de negocio. Pact JS define los contratos esperados entre el frontend y el backend, y Pact Broker almacena, versiona y verifica dichos contratos para detectar cambios incompatibles antes del despliegue.

Comparación:

Aspectos	Playwright, Testcontainers, vitest, supertest	Cypress, Docker Compose, Supertest	Postman CLI, PactJS, Pact Broker
Compatibilidad	React, Vite, TypeScript, Express, MariaDB y GitHub Actions	React, TypeScript, Express, MariaDB, Github Actions.	API's REST, TypeScript, Axios, Express y Github Actions

Configuración requerida	Instalar dependencias y navegadores de Playwright. Requiere Docker para ejecutar MariaDB temporalmente.	Configurar Cypress y un ambiente de frontend, backend y MariaDB con Docker compose.	Crear colecciones Postman, contratos pact y configurar un Pact Broker.
Tipos de pruebas	Regresión E2E, integración HTTP, base de datos, pruebas visuales y multibrowser.	Regresión E2E, intergaz, API e integración de rutas y controladores.	Regresión de API, contratos y compatibilidad entre frontend y backend.
Integración de flujo	Prueba el flujo completo desde la interfaz hasta la persistencia en MariaDB	Cypress prueba la interfaz y Supertest el backend dentro de un ambiente Docker	Postman ejecuta la API y Pact verifica los contratos entre consumidor y proveedor.
Resultados o reportes	Reportes HTML, cobertura, capturas, videos y trazas de ejecución.	Capturas, videos, registros de ejecución y resultados de pruebas HTTP.	Resultados por endpoint, historial de contratos y estado de sus verificaciones.
Documentación	Amplia documentación oficial con ejemplos para TypeScript, Docker y CI.	Documentación extensa, tutoriales y ejemplos para GitHub Actions.	Documentación completa, aunque Pact requiere conocer pruebas de contratos.
Ventajas	Cobertura integral, navegadores reales, base de datos aislada y reutiliza herramientas existentes.	Fácil de aprender, buena depuración visual y ambiente similar al de producción.	Detecta cambios incompatibles en la API y permite compartir pruebas con QA.

Limitaciones	Configuración compleja y mayor consumo de tiempo y recursos.	Las pruebas pueden ser lentas y requieren controlar los datos del ambiente.	No prueba la interfaz y añade complejidad si ambos sistemas se despliegan juntos.

Herramientas de integración continua:

1. **Github actions:**
   Esta alternativa permite implementación de CI/CD de forma nativa, está completamente integrada a los repositorios de GitHub mediantes los flujos de trabajo. Su configuración se realiza mediante los YAML ubicados específicamente dentro de la ruta, .github/workflows/ lo que permite ejecutar pruebas unitarias, de integración, regresión, E2E y procesos automatizados. La integración de flujos de desarrollo se realiza mediante PR, pushes, releases, y ramas protegidas.
   Una de sus mayores ventajas es que posee una amplia documentación, es nativa de github y tiene los registros y reportes en tiempo real desde la consola, además requiere de 0 configuración inicial y minutos mensuales gratuitos para todo tipo de repositorios. La limitación que tiene es que en proyectos muy grandes la gestión entre repositorios cruzados puede volverse dispersa.

2. **Gitlab CI/CD:**
   Esta alternativa de CI/CD se encarga de gestionar todo el ciclo de vida del desarrollo desde un único panel centralizado. Se configura mediante .gitlab-ci.yml en la raiz del repositorio y es capaz de ejecutarse en Docker. Permite pruebas unitarias, de integración, E2E y servicios auxiliares.
   Tiene la ventaja de ser uno de los pipelines más complejos y potentes de la industria, tiene versiones predefinidas para lenguajes y frameworks, sin embargo, su desventaja es que, si el repositorio no está alojado en gitlab integrarlo como herramienta de CI externa requiere configuración adicional o espejos.

3. **Jenkins:**
   Esta herramienta es una alternativa de automatización de código abierto y auto alojado, normalmente por empresas y entornos laborales mayores. Para su configuración se requiere mantener un servidor propio o contenedor Docker. Los pipelines se definen mediante un Jenkinsfile escrito en sintaxis declarativa o scripted. También, Requiere configurar webhooks y plugins específicos para integrarse con GitHub, GitLab o Bitbucket. Su documentación es amplia debido a lo ambiguo que puede ser.
   Entre sus ventajas incluye control absoluto sobre el hardware, la seguridad y los datos; costo de licencia cero; ecosistema gigantesco de más de 1,800 plugins. Entre sus

desventajas incluye alto costo de mantenimiento técnico y una configuración inicial especialmente difícil.

Comparación:

Aspectos	GitHub Actions	Gitlab CI/CD	Jenkins
Compatibilidad	Excelente para ecosistemas web modernos como React, TypeScript, Vite y backend en la nube.	Alta para proyectos full-stack, especialmente si todo el repositorio ya vive dentro de GitLab.	Universal. Compatible con prácticamente cualquier lenguaje, framework o entorno mediante plugins.
Configuración requerida	Archivos YAML en .github/workflows/. No requiere instalar servidores propios.	Archivo .gitlab-ci.yml en la raíz. Requiere configurar GitLab Runners.	Archivo Jenkinsfile. Requiere instalar, configurar y mantener un servidor propio.
Tipos de pruebas	Unitarias, integración, regresión y E2E. Permite levantar bases de datos vía contenedores.	Unitarias, integración, E2E y análisis de seguridad nativo SAST/DAST.	Ilimitadas. Maneja cualquier prueba delegando la ejecución a los nodos y plugins configurados.
Integración de flujo	Nativa con Pushes, Pull Requests y despliegues directos (ej. automatizar a GitHub Pages).	Integración directa y nativa con Merge Requests y tableros de gestión de GitLab.	Requiere configurar webhooks manualmente y plugins para conectar con el repositorio.
Resultados o reportes	Consola en tiempo real, descarga de artefactos y	Pestaña dedicada en la interfaz con lectura nativa de reportes (ej.	Altamente personalizable con gráficos detallados a

	resúmenes visuales en los Pull Requests.	JUnit) y cobertura de código.	través de plugins adicionales.
Documentación	Extensa y respaldada por un inmenso Marketplace con miles de acciones listas para usar.	Muy estructurada y clara, con múltiples plantillas oficiales predefinidas.	Enorme pero a veces fragmentada o desactualizada por la gran cantidad de versiones de plugins.
Ventajas	Cero mantenimiento de infraestructura e integración perfecta si el código ya está alojado ahí.	Todo el ciclo de vida del repositorio, CI/CD y tickets unificado en una sola plataforma potente.	Totalmente gratuito, máxima personalización y control absoluto sobre la seguridad y el hardware.
Limitaciones	Límites de minutos gratuitos en repositorios privados; manejo de secretos complejo en proyectos muy grandes.	Configuración adicional y menos fluida si el repositorio principal está en otra plataforma.	Alto costo de tiempo en mantenimiento técnico y dependiente de plugins que pueden romperse.

Selección de tecnologías:

Elección de herramienta de integración y regresión:

La elección de Playwright, Testcontainers, Vitest y Supertest se fundamenta en su capacidad para automatizar pruebas de regresión e integración en todas las capas del sistema. A diferencia de alternativas como Cypress, que se concentra principalmente en la interacción con la interfaz, Playwright permite validar los flujos principales en distintos navegadores reales, incluyendo autenticación, reservas, asignación de tareas y notificaciones. Por su parte, Vitest y Supertest se integran naturalmente con el backend desarrollado en Express y TypeScript, permitiendo comprobar rutas, middlewares, controladores y respuestas HTTP. Testcontainers complementa esta estrategia mediante la creación automática de una instancia temporal y aislada de MariaDB, con la cual es posible validar consultas, transacciones y persistencia real sin modificar los datos de desarrollo o producción. Esta combinación también ofrece una cobertura más completa que Postman CLI y Pact JS, debido a que no se limita a verificar endpoints o contratos, sino que comprueba el funcionamiento integral desde la interfaz hasta la base de datos. Finalmente, su compatibilidad con Docker y GitHub Actions facilita la ejecución automática de las pruebas en cada pull request, permitiendo detectar regresiones antes de integrar cambios en la rama principal; por estas razones, se eligió la combinación de Playwright, Testcontainers, Vitest y Supertest.

Elección de herramienta de integración contínua:

La elección de GitHub Actions se fundamenta en su integración nativa y la eliminación total de la carga de infraestructura. A diferencia de herramientas clásicas como Jenkins, que exigen instalar y mantener servidores propios, Actions opera como un servicio completamente gestionado en la nube, lo que permite concentrar el esfuerzo exclusivamente en la automatización de las pruebas. Al estar el código alojado en GitHub, se evita la fricción de configurar webhooks externos o sincronizaciones manuales que requerirían opciones como GitLab CI/CD, logrando que los workflows se ejecuten de forma natural ante cada push o pull request. Esta herramienta resulta ideal para nuestro flujo de desarrollo, ya que facilita la configuración de entornos para ecosistemas modernos basados en React, TypeScript y Vite, e incluso simplifica despliegues automatizados. Finalmente, su capacidad para exponer los resultados directamente en la interfaz y bloquear la integración de código si una prueba falla, asegura que cualquier regresión sea detectada y corregida de inmediato antes de afectar la rama principal, por esa razón, se eligió github actions.

Análisis de resultados

Tiempo y esfuerzo de automatización

La implementación de las pruebas automatizadas requirió un esfuerzo inicial para configurar las herramientas, preparar los datos y definir los escenarios de prueba. El trabajo se concentró en los procesos principales de la plataforma administrativa de la parroquia, como la autenticación, la gestión de reservas, la asignación de tareas y los cambios de turno.

La automatización permite repetir estas verificaciones después de modificar el sistema, reduciendo el esfuerzo de revisión manual. No se cuenta con un registro de horas que permita cuantificar el ahorro, pero disponer de pruebas reutilizables facilita la detección temprana de errores y aporta mayor consistencia a la validación.

Utilidad de las herramientas y configuración

Playwright permite verificar los flujos de regresión desde la perspectiva del usuario, comprobando la navegación, los formularios y los resultados de las operaciones. Vitest organiza y ejecuta las pruebas del backend, mientras que Supertest permite comprobar las respuestas de los endpoints mediante solicitudes HTTP. Testcontainers proporciona una base de datos temporal y aislada para validar la interacción con MariaDB sin afectar los datos del entorno habitual.

GitHub Actions complementa esta estrategia mediante la ejecución automática de las pruebas durante la integración de cambios, facilitando la identificación de fallos antes de incorporar nuevas modificaciones.

Durante la implementación no se presentaron inconvenientes relevantes en la configuración de las herramientas. Los ajustes necesarios correspondieron a la preparación habitual del entorno, las dependencias y los datos de prueba, por lo que no fue necesario aplicar soluciones extraordinarias.

Fallos detectables y límites de la estrategia

La estrategia implementada permite detectar regresiones en los escenarios cubiertos, errores de validación, respuestas incorrectas de la API y problemas de persistencia. También permite comprobar restricciones de acceso y reglas de negocio, como evitar reservas con horarios superpuestos o verificar que las operaciones se realicen con los permisos correspondientes.

Sin embargo, su alcance depende de los casos definidos. Las funcionalidades y combinaciones de datos que no se incluyan pueden contener errores que pasen inadvertidos. Asimismo, estas pruebas no sustituyen las evaluaciones de usabilidad, las pruebas de carga y estrés ni las revisiones especializadas de seguridad. Tampoco garantizan que el comportamiento sea idéntico en todos los entornos de producción.

Conclusión

Se mantendrían las herramientas seleccionadas porque cubren aspectos complementarios del proyecto: la interacción del usuario, el comportamiento del backend, la comunicación con la base de datos y la ejecución automática de las verificaciones.

La experiencia de implementación fue favorable, ya que la configuración se completó sin inconvenientes relevantes. La estrategia proporciona una base para comprobar que los cambios preserven las funcionalidades existentes. Para conservar su utilidad, será necesario actualizar las pruebas conforme evolucione el sistema y ampliar la cobertura de los procesos más importantes.

REPOSITORIO DE GITHUB

https://github.com/hmndzzl/Software_Proyecto/tree/develop

LINK DE VIDEO:

https://youtu.be/GgDYLNwdPzk